Detection Engineering Report

Enterprise DevSecOps Security Lab

Prepared By	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Document Type	Runtime Detection Engineering and Security Observability
Environment	Private Ubuntu VM, Kind Kubernetes, Falco, Prometheus, Grafana
Classification	Portfolio / Private Lab Evidence

Document Intent
This professional portfolio artifact describes the design, implementation, validation, and operational usage of runtime detection, alerting, dashboards, or response artifacts created inside the private Enterprise DevSecOps lab.

Coverage Area	Included in this document
Runtime detection	Falco event logic, validation evidence, and triage workflow
Security observability	Prometheus metrics, Grafana dashboarding, alert health checks
Response readiness	Analyst actions, containment commands, retest criteria, evidence handling
Portfolio value	Professional documentation showing detection engineering, SOC process, and cloud-native security maturity

Lab boundary: no external systems were targeted. All attack simulations, detections, and response workflows were performed inside an isolated private VM and Kind Kubernetes environment.

Executive Summary

This detection engineering report documents how the Enterprise DevSecOps lab detects Kubernetes runtime threats using Falco, exposes runtime security signals as Prometheus metrics, and visualizes detections inside Grafana. The goal is to show an enterprise-style detection lifecycle: threat hypothesis, telemetry source, detection logic, validation, alerting, triage, containment, and evidence preservation.

The detection coverage focuses on high-impact cloud-native behaviors: sensitive file reads, privileged pod activity, hostPath abuse, node filesystem access patterns, Falco kernel event drops, and Falco output queue drops. These signals support incident response for container escape attempts, RBAC abuse, and malicious workload behavior.

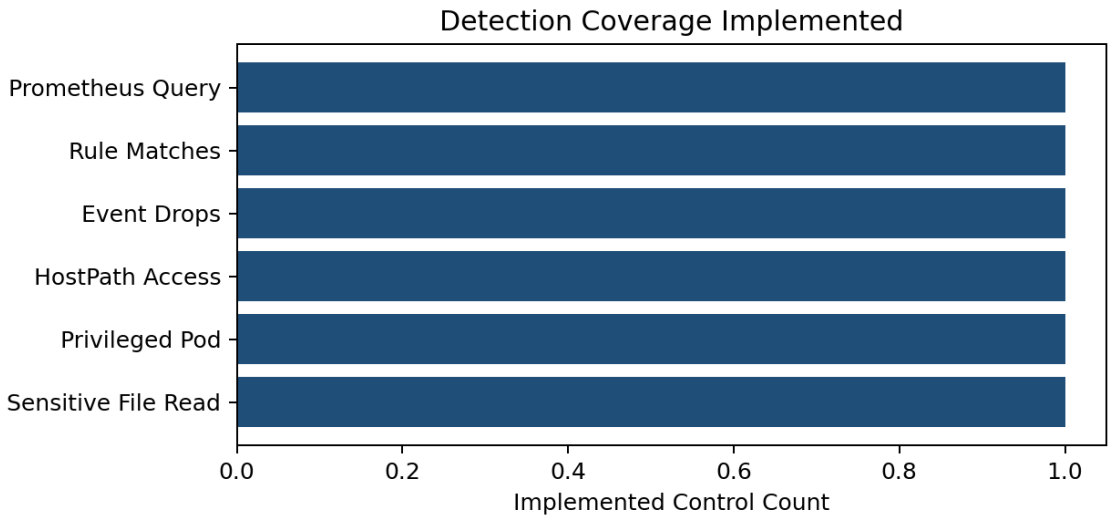
Professional Outcome
The project demonstrates practical detection engineering beyond tool installation: signals are mapped to attack scenarios, PromQL queries are tied to alert rules, dashboards are mapped to operational decisions, and each detection has validation and response criteria.

Scope and Architecture

Layer	Implemented Control	Evidence / Artifact
Runtime telemetry	Falco syscall and Kubernetes-aware detection	Falco running in falco namespace; runtime alert logs
Event forwarding	Falco/Falcosidekick metrics endpoint	Falco metrics queried by Prometheus
Metrics storage	Prometheus via kube-prometheus-stack	Prometheus targets and query screenshots
Visualization	Grafana custom dashboard	falco-security-dashboard.json
Alerting	PrometheusRule object	falco-security-prometheus-rules.yaml
Response	Incident runbook and triage workflow	Runbook document and commands

Detection Pipeline

The detection pipeline starts with Falco receiving kernel and workload activity, such as file access or process execution. Falco evaluates rules and emits events with context such as rule name, priority, process, container image, pod name, and namespace. Falco/Falcosidekick metrics are scraped by Prometheus. Grafana dashboards visualize rule matches, priority trends, top triggered rules, and health signals such as dropped kernel events.



Detection coverage implemented across runtime, alert health, and validation paths.

Threat Hypotheses

Threat Hypothesis	Telemetry Needed	Detection Signal	Response Decision
A container attempts to read sensitive host or container files	File open events, process name, container context	Sensitive file opened for reading by non-trusted program	Confirm pod, namespace, image, and workload owner; isolate if unauthorized
An attacker runs a privileged pod to access the node filesystem	Pod security context, hostPath mounts, process execution	Privileged pod behavior plus sensitive file access	Delete pod, enforce restricted Pod Security, review admission controls
Falco detection quality is degraded	Kernel event counters and output queue metrics	Kernel event drops or output queue drops > 0	Treat as monitoring integrity incident; reduce event load or tune Falco resources
Unexpected spike in runtime detections	Rule match counters by priority and rule name	falcosecurity_falco_rules_matches_total increase	Triage high-priority rule, correlate with deployment changes

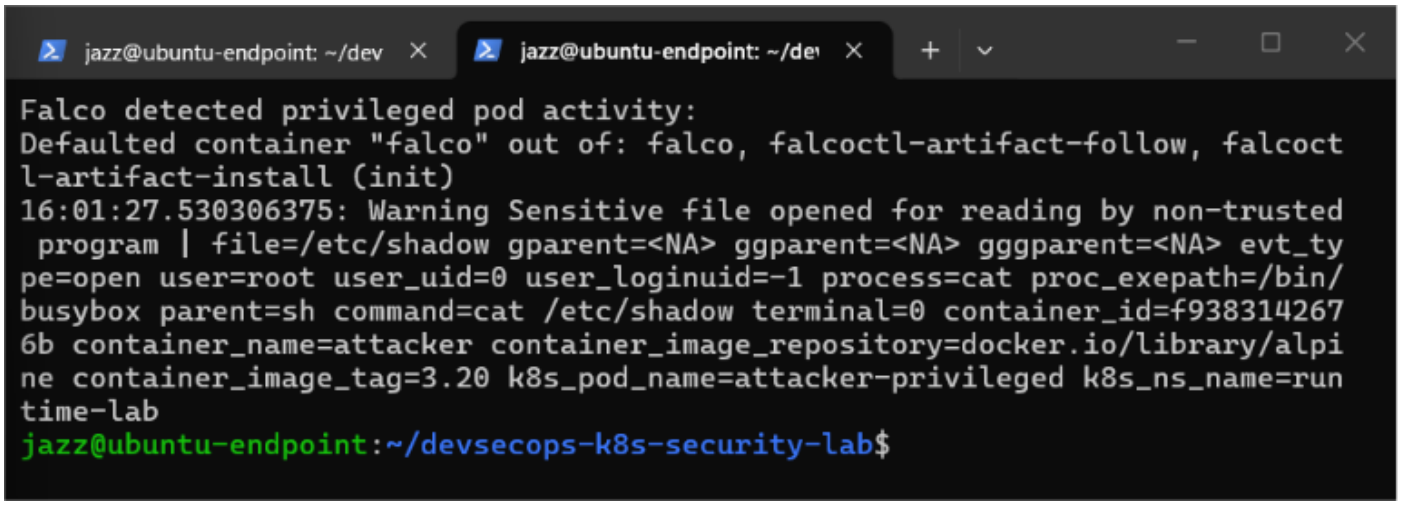
Detection Logic and Metrics

Falco rules provide the runtime detection logic. Falco rule priorities represent the seriousness of a violation and can be emitted in JSON output and metrics. Falco fields can be used in rule conditions and outputs, allowing detections to include Kubernetes context such as pod and namespace when available. Prometheus alert rules then evaluate time-windowed increases in Falco metrics.

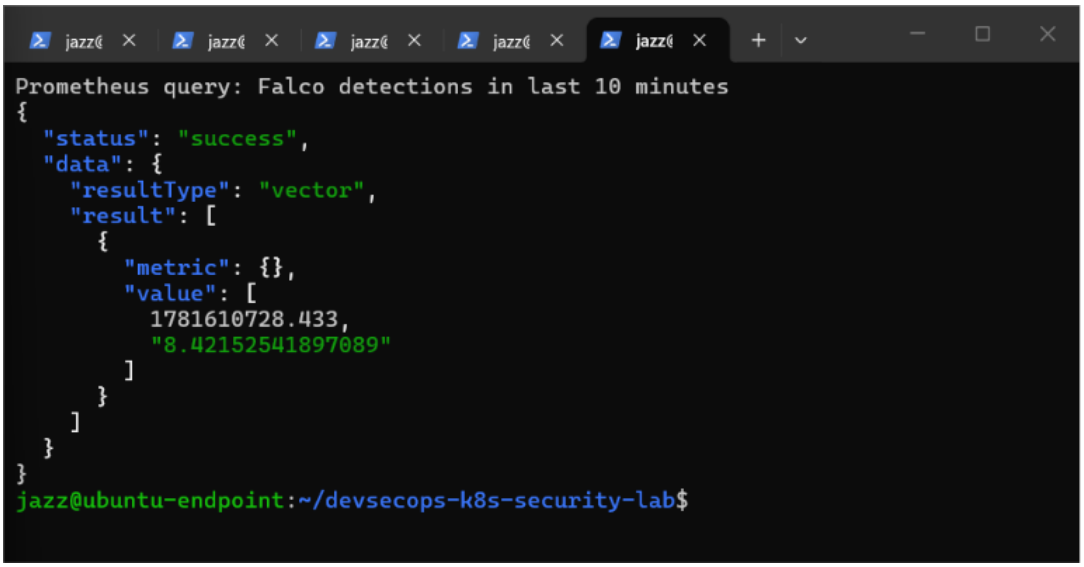
```
# Primary PromQL signal used for runtime detections
sum by (rule_name, priority, source) (
  increase(falcosecurity_falco_rules_matches_total[5m])
) > 0

# Alert integrity signals
sum(increase(falcosecurity_scap_n_drops_total[5m])) > 0
sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[5m])) > 0
```

Validation Evidence

Evidence: Falco pods installed and running in the private Kubernetes lab.



Evidence: Falco detected sensitive file activity from a privileged attacker pod.



Evidence: Prometheus query returned Falco rule match metrics after test events.

Detection Quality Assessment

Quality Dimension	Assessment	Reasoning
Coverage	Strong for lab runtime scenarios	Covers privileged pod behavior, sensitive files, rule matches, and health drops
Signal context	Strong	Events include process, user, container, pod, namespace, and image fields where available
False positive risk	Medium	Sensitive file rules can trigger during legitimate diagnostics; tuning should allowlist trusted admin jobs
Operational readiness	Good	Dashboard and Prometheus rules support detection visibility and alerting
Production maturity	Moderate	Needs alert routing, owner mapping, and baseline tuning before production use

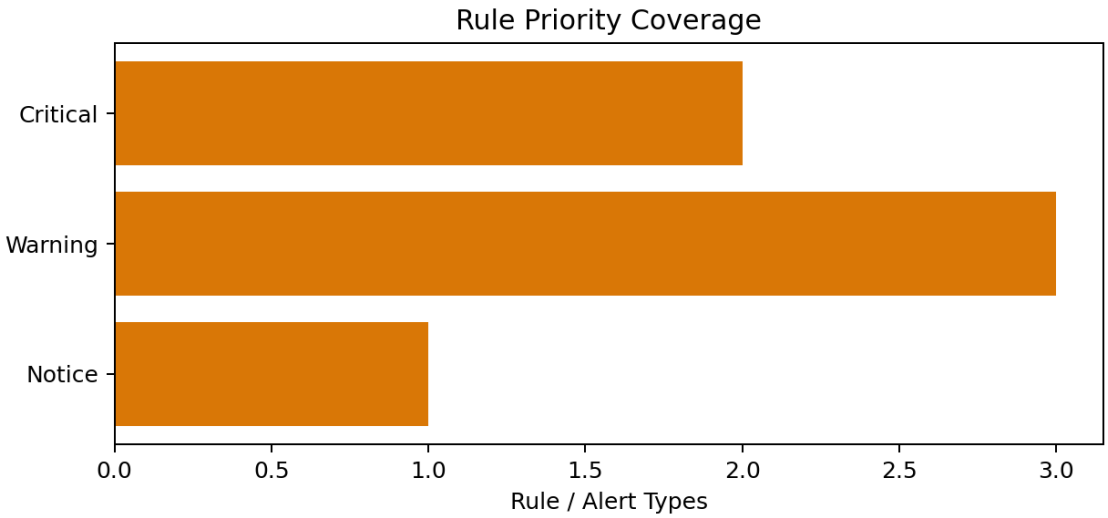
Recommended Improvements

- Route critical Falco alerts to a ticketing or incident channel with pod, namespace, image, node, and owner metadata.
- Create allowlists for known backup, scanning, and admin workloads to reduce false positives.
- Add runbook links to Prometheus rule annotations so analysts can follow a consistent response process.

- Export Falco events to long-term storage for investigation and retention.
- Define service ownership labels and map each namespace to an escalation contact.

MITRE ATT&CK / Cloud-Native Mapping

Scenario	ATT&CK-style Behavior	Detection Control
Privileged pod with hostPath	Privilege escalation / escape from container boundary	Falco privileged activity and Pod Security validation
Sensitive file read	Credential and secret discovery	Falco sensitive file access detection
Unexpected shell execution	Execution and discovery	Falco shell and process events
Dropped events	Defense evasion / telemetry loss risk	Prometheus alert on Falco drop counters



Rule and alert priority coverage used by the lab detection strategy.

References

Falco rules basic elements - https://falco.org/docs/concepts/rules/basic-elements/

Falco supported fields for rule conditions and outputs - https://falco.org/docs/reference/rules/supported-fields/

Falco metrics - https://falco.org/docs/concepts/metrics/

Falco alert forwarding and JSON outputs - https://falco.org/docs/concepts/outputs/forwarding/

Prometheus alerting rules - https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

Grafana dashboard JSON model - https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/view-dashboard-json-model/

Grafana dashboard import workflow - https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/import-dashboards/